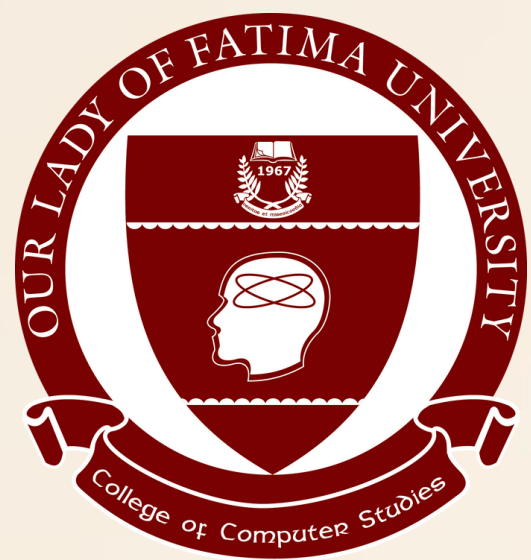




Module 6

Programming Concepts

ITCL112 - INTRODUCTION TO COMPUTING W/ LAB

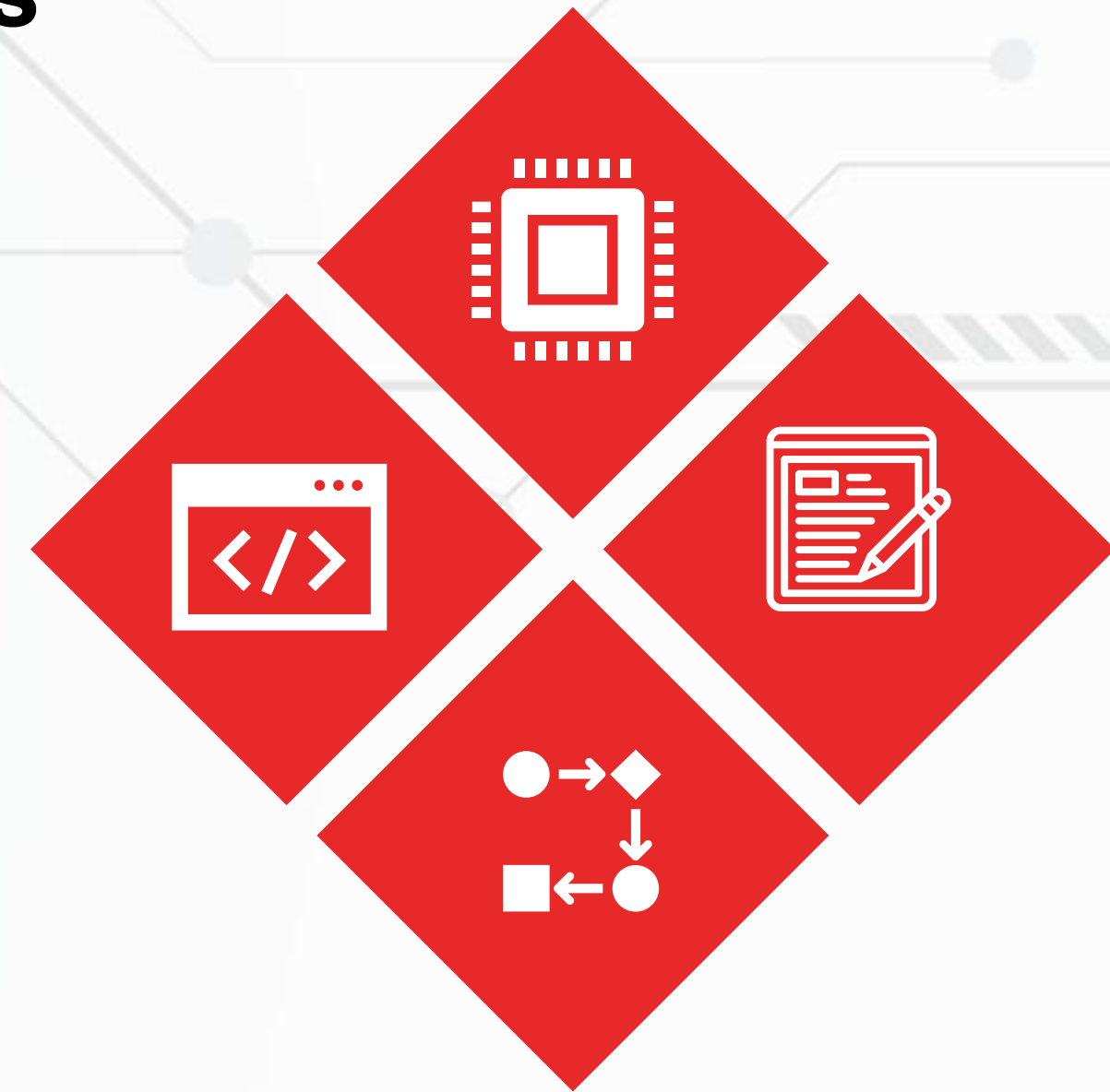
INTRODUCTION



Programming is the art and science of instructing computers to perform tasks. At its core, it's about solving problems using logic, creativity, and precision. Whether you're building a simple calculator or a complex AI system, programming concepts form the foundation of every digital solution.

MODULE 6

1. What is programming?
2. Algorithm and Flowchart
3. Introduction to Programming Languages



What is programming?

Programming is the process of creating a set of instructions that a computer can follow to perform specific tasks. These instructions are written in a programming language that computers can understand and execute.



Purpose of Programming



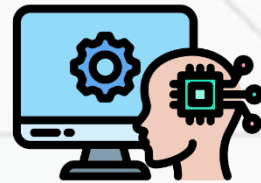
Automate task

Replaces repetitive manual work with computer-executed processes



Solve problem efficiently

Provides solutions to real-world problems through logical steps



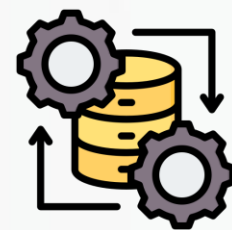
Human-Computer Interaction

Computers only understand machine language (0s and 1s). Programming allows humans to “translate” ideas into instructions computers can execute.



Create software, apps and systems

From mobile apps and websites to video games and enterprise systems, all software exists because of programming.



Data Processing and Analysis

Programming helps process large amounts of data quickly for business decisions, scientific research, or machine learning.



Innovation & Technology Advancement

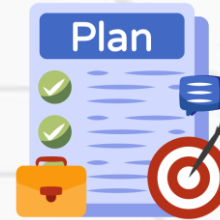
Enables new technologies like AI, robotics, self-driving cars, and smart devices.

Programming Workflow



Problem Analysis

- Understand what needs to be solved.
- Define inputs, processes, and expected outputs.



Planning & Design

- Create a solution approach using algorithms or flowcharts.
- Decide the logic before writing code.



Coding (Implementation)

- Write the program using a programming language



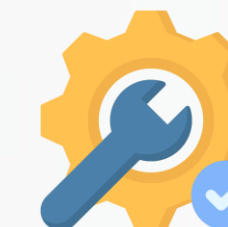
Testing & Debugging

- Run the program to check for errors (bugs).
- Fix logical, syntax, or runtime errors.



Execution & Deployment

- Execute the final program and use it in the real environment.
- Deploy it for users (e.g., install as an app, publish on a website).



Maintenance & Updates

- Improve performance, fix new bugs, or add new features over time.

Advantages & Disadvantages



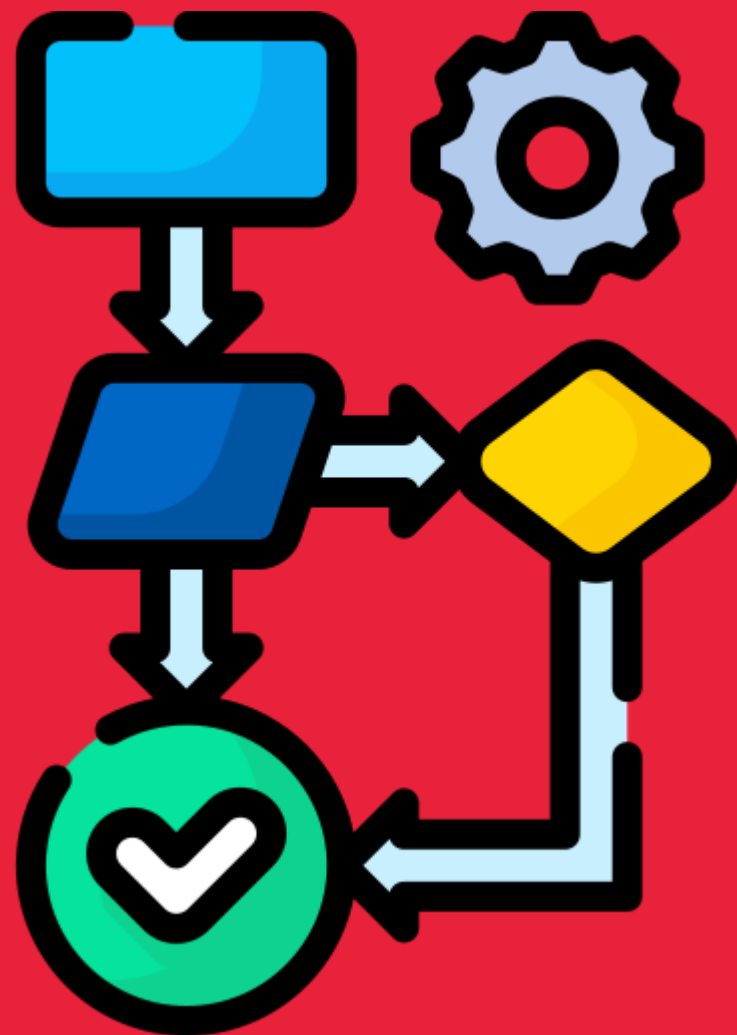
Advantages

1. **Increased Productivity:** Programming languages provide a set of abstractions that allow developers to write code more quickly and efficiently.
2. **Portability:** Programs written in a high-level programming language can run on many different operating systems and platforms.
3. **Readability:** Well-designed programming languages can make code more readable and easier to understand for both the original author and other developers.
4. **Large Community:** Many programming languages have large communities of users and developers, which can provide support, libraries, and tools.

Disadvantages

1. **Complexity:** Some programming languages can be complex and difficult to learn, especially for beginners.
2. **Performance:** Programs written in high-level programming languages can run slower than programs written in lower-level languages.
3. **Limited Functionality:** Some programming languages may not have built-in support for certain types of tasks or may require additional libraries to perform certain functions.
4. **Fragmentation:** There are many different programming languages, which can lead to fragmentation and make it difficult to share code and collaborate with other developers.

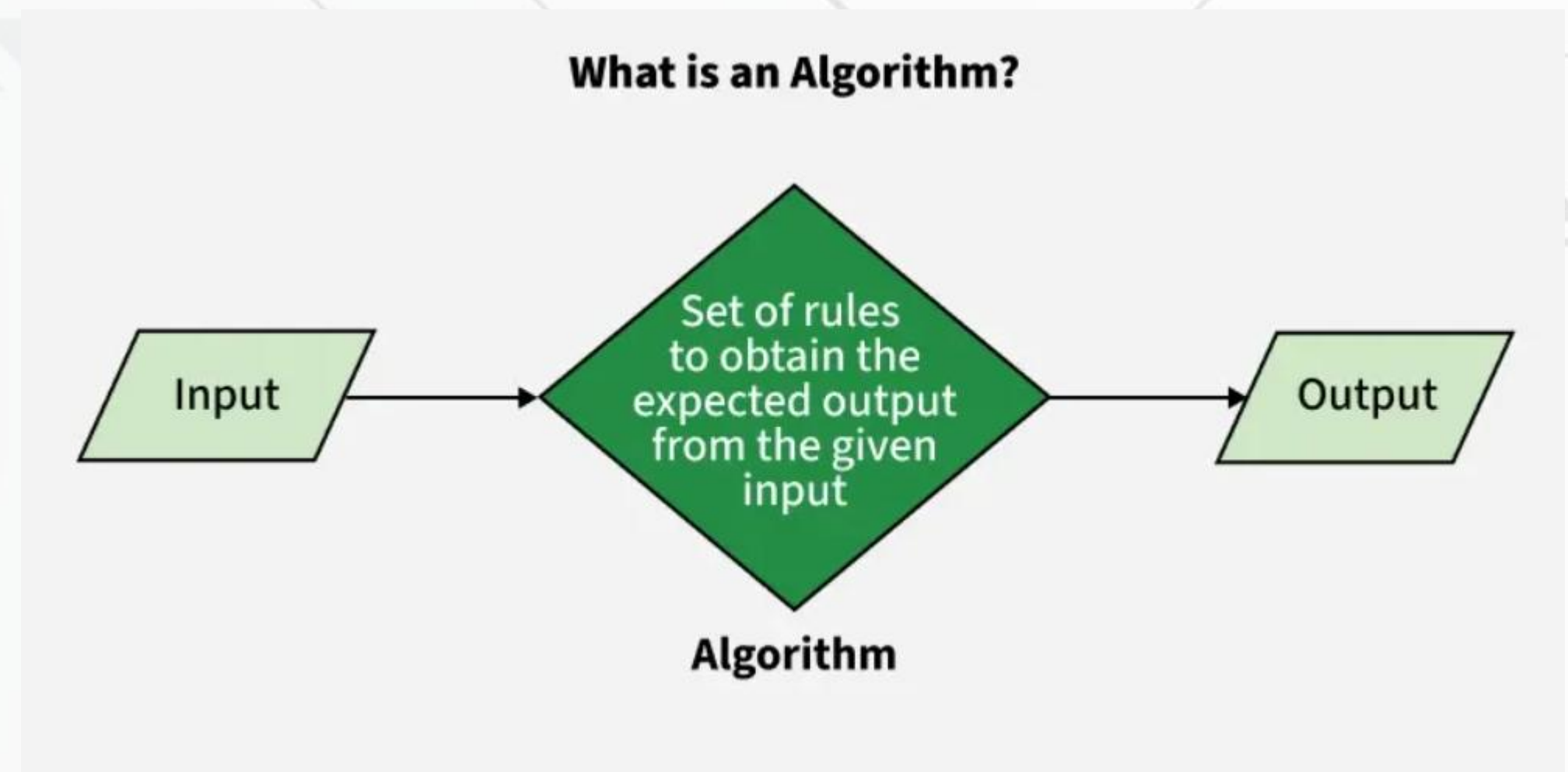
Algorithm & Flowchart



Algorithm

It is a list of instructions specifying a precise description of a step-by-step process that terminates after a finite number of steps for solving an algorithm problem producing the correct answer in the end.

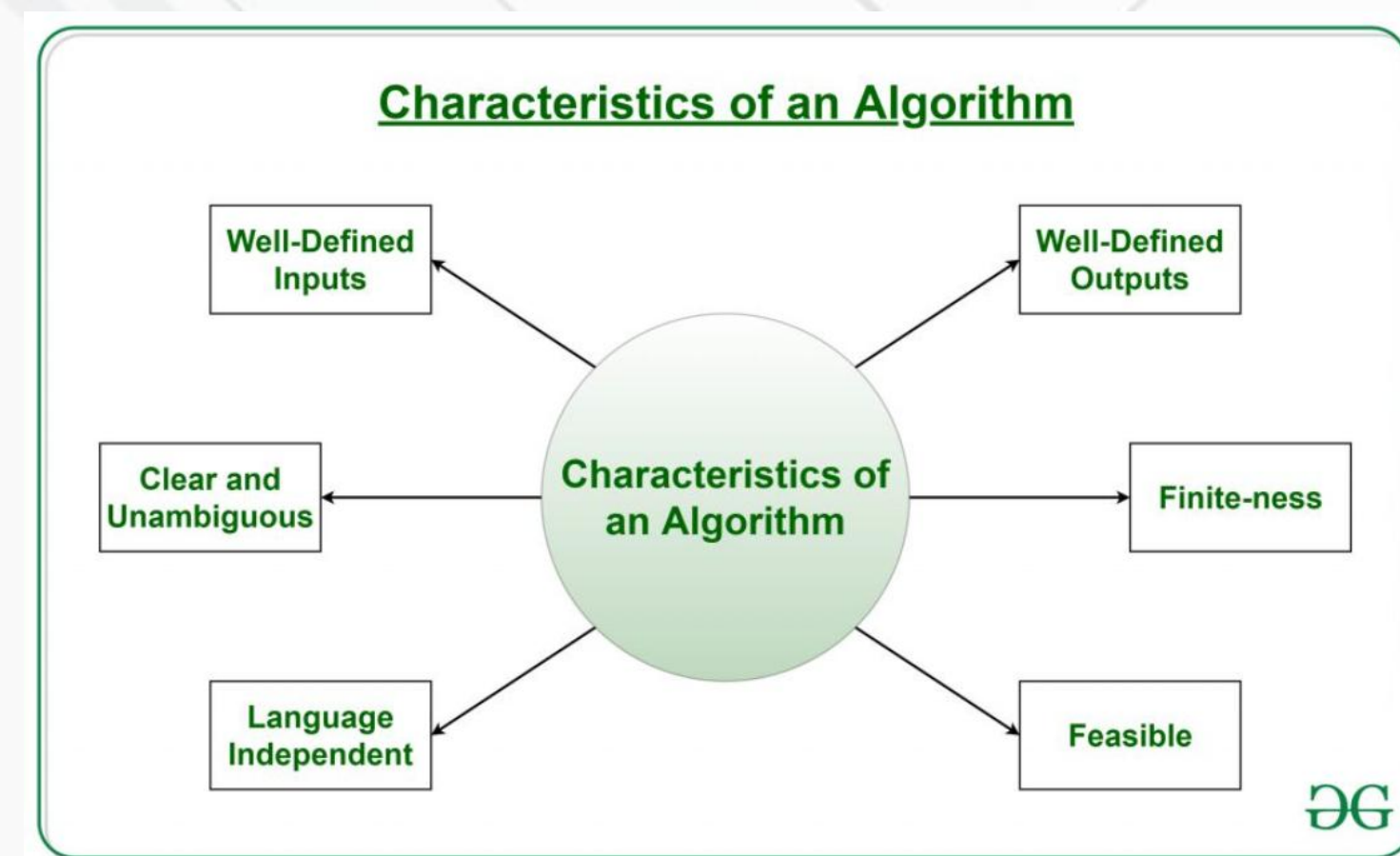
It is a recipe for solving problems.



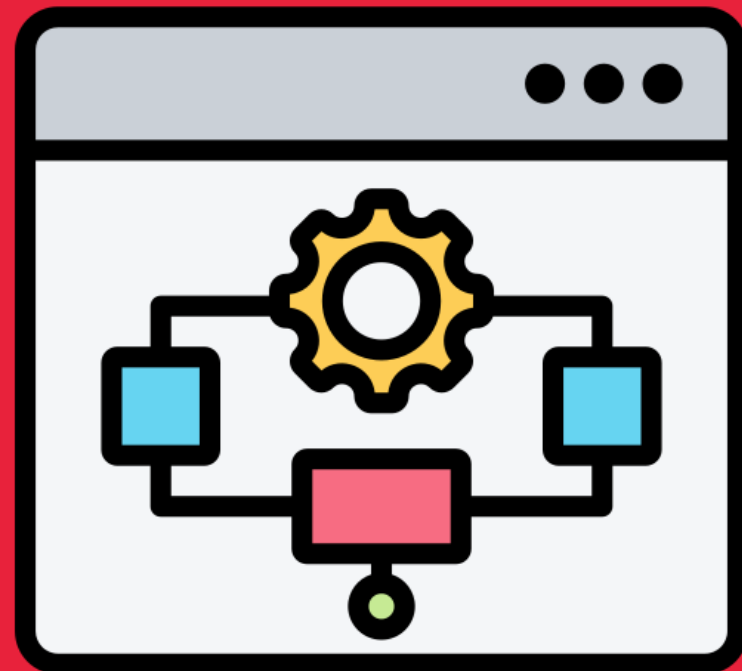
Characteristics of an Algorithm



- **Clear and Unambiguous:** The algorithm should be unambiguous. Each of its steps should be clear in all aspects and must lead to only one meaning.
- **Well-Defined Inputs:** If an algorithm says to take inputs, it should be well-defined inputs. It may or may not take input.
- **Well-Defined Outputs:** The algorithm must clearly define what output will be yielded and it should be well-defined as well. It should produce at least 1 output.
- **Finite-ness:** The algorithm must be finite, i.e. it should terminate after a finite time.
- **Feasible:** The algorithm must be simple, generic, and practical, such that it can be executed with the available resources. It must not contain some future technology or anything.
- **Language Independent:** The Algorithm designed must be language-independent, i.e. it must be just plain instructions that can be implemented in any language, and yet the output will be the same, as expected.



Properties of Algorithm



Finiteness

There is an exact number of steps to be taken and has an end.

Absence of Ambiguity

Means that every instruction is precisely described and clearly specified.

Sequence of Execution

Instructions are performed from top to bottom.

Input and Output

Defined the unknowns of the problem is specified and with the expected outcome.

Effectiveness

The solution prescribed is guaranteed to give a correct answer and that the specified process is faithfully carried out.

Scope of Definition

Applies to a specific problem or class of problem.

Methods of Specifying Algorithm

Pseudocode

A Pseudocode is defined as a step-by-step description of an algorithm. Pseudocode does not use any programming language in its representation instead it uses the simple English language text as it is intended for human understanding rather than machine reading.

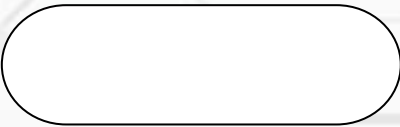
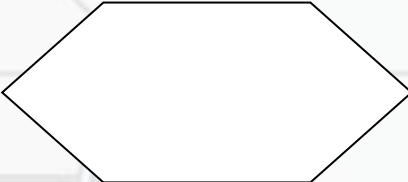
Flowchart

Flowcharts are the visual representations of an algorithm or a process. Flowcharts use symbols/shapes like arrows, rectangles, and diamonds to properly explain the sequence of steps involved in the algorithm or process. Flowcharts have their use cases in various fields such as software development, business process modeling, and engineering.

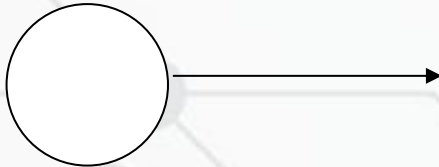
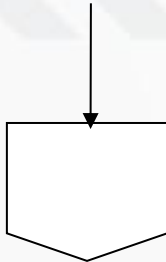
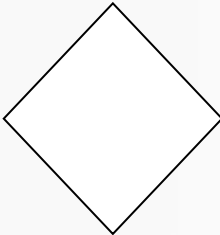
Flowcharting Guidelines

1. The flowchart should flow from top to bottom
2. If the chart becomes complex, utilize connecting blocks
3. Avoid intersecting flow lines
4. Use meaningful description in the symbol

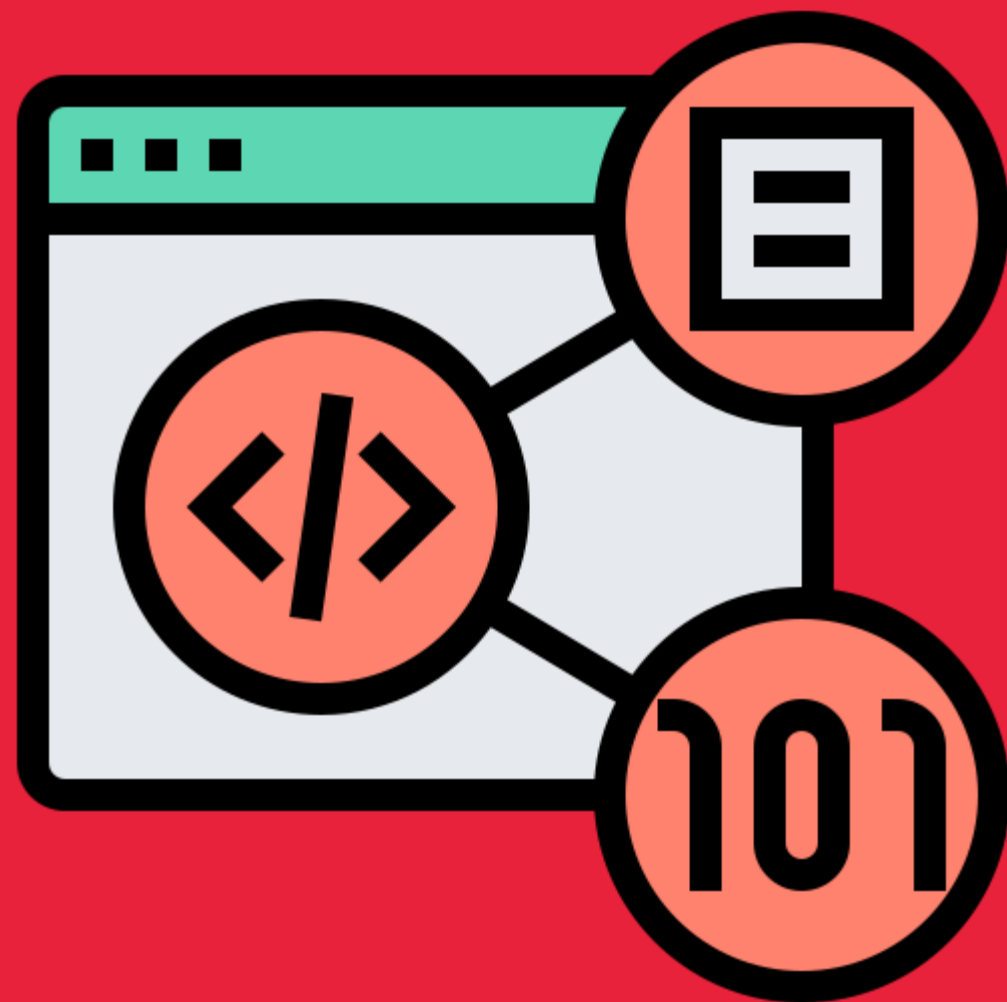
Basic Flowcharting Symbols

SYMBOL	NAME	DESCRIPTION
	TERMINAL	Defines the starting and ending point of a flowchart.
	INITIALIZATION	The preparation or initialization of memory space for data processing.
	INPUT/OUTPUT	The inputting of data for processing, and printing out of processed data.
	PROCESS	Manipulation of data(assignments and mathematical computations)

Basic Flowcharting Symbols

SYMBOL	NAME	DESCRIPTION
	FLOW LINES	Defines logical sequence of the program. Its points to the next symbol to be performed
	ON-PAGE CONNECTOR	Connects to the flowchart to avoid spaghetti connection on the same page
	OFF-PAGE CONNECTOR	Connects the flowchart on different page to avoid spaghetti connection
	DECISION	Process conditions using relational operators. Used for trapping and filtering data.

Introduction to Programming Languages



A programming language is a set of instructions and syntax used to create software programs.

Syntax

The specific rules and structure used to write code

Data Type

The type of values that can be stored in a program (Eg. Int, char, bool)

Variables

Named memory locations that can store values.

Operators

Symbols used to perform operations on values (E.g. Addition, Subtraction)

Control Structures

Statements used to control the flow of a program (E.g. If-else, loop, functions)

Libraries and Frameworks

Collections of pre-written code that can be used to perform common tasks and speed up development.

Paradigms

The programming style or philosophy used in the language (E.g. Procedural, OOP)

Hierarchy of Programming Languages



Low-Level Programming Languages

The computer understandable language

Machine Language and Assembly Language

High-Level Programming Languages

Nearly human languages that are more complex

Python, Java, C, etc.



Most Popular Programming Languages



- ✓ C
- ✓ Python
- ✓ C++
- ✓ Java
- ✓ SCALA
- ✓ C#
- ✓ R
- ✓ Ruby
- ✓ Go
- ✓ Swift
- ✓ JavaScript



Characteristics of a Programming Language

- 1. A programming language must be simple, easy to learn and use, have good readability, and be human recognizable.**
- 2. Abstraction is a must-have Characteristics for a programming language in which the ability to define the complex structure and then its degree of usability comes.**
- 3. A portable programming language is always preferred.**
- 4. Programming language's efficiency must be high so that it can be easily converted into a machine code and its execution consumes little space in memory.**
- 5. A programming language should be well structured and documented so that it is suitable for application development.**
- 6. Necessary tools for the development, debugging, testing, maintenance of a program must be provided by a programming language.**
- 7. A programming language should provide a single environment known as Integrated Development Environment(IDE).**
- 8. A programming language must be consistent in terms of syntax and semantics.**

Tips for Learning New Programming Languages

Start with Fundamentals

Code Daily

Work on projects

Read the Documentation

Join Online Communities

Learn from others

Practice Debugging